Reviewer Pack — Minimal Hodge Index and DPLL Proof Path Length Lower Bound

Entry eb7c53ea343a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 08:36:40 UTC

Minimal Hodge Index and DPLL Proof Path Length Lower Bound

Entry ID: `eb7c53ea343a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 08:36:40 UTC

1. Conjecture

Field A (mathematical branch): Hodge Theory **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

> For every CNF formula φ with n variables, the minimal Hodge index of its associated algebraic variety is linearly correlated with its DPLL proof path length, such that H(φ) = Θ(P(φ)) where P(φ) represents the DPLL proof path length and H(φ) is a function of the complexity of the algebraic variety associated with φ.

Rationale (proposer's reasoning):

> Hodge theory, which studies the topology of complex algebraic varieties, could provide insights into the structure of the search space in Boolean satisfiability problems. The minimal Hodge index captures the complexity of the underlying geometric object, which might correlate with the difficulty of finding a satisfying assignment for the formula.

Taxonomy category: `HODGE_DPLL` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f017beb2e64235fc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the 30 generated CNF formulas φ have a correlation coefficient between their minimal Hodge index and DPLL proof path length greater than or equal to 0.7, with an average absolute difference in ranks less than or equal to 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge index" AND "DPLL proof path length"` - `"algebraic variety" IN BOOLEAN Satisfiability` - `"minimal Hodge index" related TO "Boolean satisfiability"`

Top relevant hits considered: - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2206.00903v2] Satisfiability of Quantified Boolean Announcements - [http://arxiv.org/abs/1908.01979v1] Non-Invasive Reverse Engineering of Finite State Machines Using Power Analysis and Boolean Satisfiability - [http://arxiv.org/abs/1510.02682v8] Classical and Quantum Algorithms for the Boolean Satisfiability Problem - [http://arxiv.org/abs/1602.06867v4] Lower bound for the Complexity of the Boolean Satisfiability Problem - [http://arxiv.org/abs/1404.6682v1] Sampling Techniques for Boolean Satisfiability - [s2:f861e08f5a76206f0f878d393fdfb40bdb53127e] Automata, languages and programming : 31st International Colloquium, ICALP 2004, Turku, Finland, July 12-16, 2004 : proc

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(n):
                if j != i:
                    factor = Fraction(A[j][i], A[i][i])
                    A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
        return A

    def determinant(A):
        m, n = len(A), len(A[0])
        if m != n:
            raise ValueError("Matrix must be square")
        det = Fraction(1)
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
```

```
            A[i], A[max_row] = A[max_row], A[i]
            det *= A[i][i]
            if det == 0:
                return 0
            for j in range(n):
                if j != i:
                    factor = Fraction(A[j][i], A[i][i])
                    A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]
    return det

def grothendieck_witt_class_mod_2(matrix):
    det = determinant(matrix)
    if det == 0:
        return 0
    elif det > 0:
        return 1
    else:
        return -1

def dpll_proof_path_length(cnf):
    stack = []
    for clause in cnf:
        if not any(var in stack or -var in stack for var in clause):
            stack.append(random.choice(clause))
    return len(stack)

def hodge_index(n):
    # Simplified Hodge index calculation based on n
    return n

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(-n, -1) for _ in range(random.randint(1, n))]
        cnf.append(clause)
    return cnf

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5):
        cnf = generate_cnf(n, random.randint(1, n))
        hodge = grothendieck_witt_class_mod_2([[var % 2 for var in clause] for clause in cnf])
        dpll_path_length = dpll_proof_path_length(cnf)
        results.append((hodge, dpll_path_length))

if not results:
    return {
        "metric_name": "Hodge Index vs DPLL Path Length",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }
```

```
    hodge_values = [h for h, _ in results]
    dpll_path_lengths = [d for _, d in results]

    def rank(x):
        return sorted(x).index(x)

    hodge_ranks = [rank(h) for h in hodge_values]
    dpll_ranks = [rank(d) for d in dpll_path_lengths]

    correlation_coefficient = sum((h - mean_hodge) * (d - mean_dpll) for h, d in zip(hodge_values,
    mean_absolute_difference = sum(abs(hr - dr) for hr, dr in zip(hodge_ranks, dpll_ranks)) / len(

    return {
        "metric_name": "Hodge Index vs DPLL Path Length",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": abs(correlation_coefficient) >= 0.7 and mean_absolute_difference <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE <reason>")
    else:
        mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
        std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in res
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_frac
        else:
            first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), N
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}"
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b12a25ea.py", line 123, in <module>
    trial_result = run_trial(seed)
```

```
                  ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b12a25ea.py", line 82, in run_trial
    hodge = grothendieck_witt_class_mod_2([[var % 2 for var in clause] for clause in cnf])
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b12a25ea.py", line 50, in grothendieck_
    det = determinant(matrix)
          ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b12a25ea.py", line 35, in determinant
    raise ValueError("Matrix must be square")
ValueError: Matrix must be square
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the correlation coefficient between the minimal Hodge index and DPLL proof path length as required by the support condition. | next: Investigate the cause of the crash in the test code and attempt to run the test again to gather the necessary data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14071
2	preregistration	ollama_remote	glm4:latest	0	0	9244
3	novelty	ollama_remote	glm4:latest	0	0	8312
4	novelty	ollama_remote	glm4:latest	0	0	9844
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20363
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16984
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14728
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51862
9	judge	ollama_remote	glm4:latest	0	0	64534

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 209942 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/eb7c53ea343a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/eb7c53ea343a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/eb7c53ea343a.tar.gz` (if generated)